Practical 1: Generate a First RetailPulse Report

Related topic: Introduction to IBM Cognos Analytics – Reporting

Objective

Navigate the Cognos Analytics Reporting environment and generate a running list report against the RetailPulse Sales package.

Materials required

IBM Cognos Analytics access, RetailPulse Sales package, a personal content folder to save into.

Step-by-step procedure

1. Open IBM Cognos Analytics and select **New > Report** from the welcome screen.
2. Choose the **RetailPulse Sales** package as the data source when prompted.
3. Pick the **List** template from the template gallery to start with a simple layout.
4. From the **Source** tab, drag **Region**, **Product Category**, and **Sales Amount** onto the canvas.
5. Click **Run** in the toolbar to generate the report and preview live RetailPulse data.
6. Save the report as **"RetailPulse – First Sales Report"** in your personal folder (**Save as > My content**).

Expected output (illustrative)

RetailPulse – First Sales Report | Run: 06 October 2026

Region	Product Category	Sales Amount
East	Electronics	429,000
East	Apparel	95,050
East	Home & Garden	58,550
North	Electronics	378,000
North	Apparel	76,400
North	Home & Garden	70,000
…	…	…

Mock-up: truncated preview of the three-column list; rows are not grouped in this first report, so labels repeat per row.

Expected outcome
Report canvas shows live rows (not placeholder text); the three columns are labelled Region, Product Category, Sales Amount; the report is saved and re-opens from *My content* with the same data connection.

Practical 2: Build a Grouped RetailPulse Sales List Report

Related topic: Create List Reports

Objective

Construct a list report, group it meaningfully, format its columns, and add a summarizing footer.

Materials required

IBM Cognos Analytics access, RetailPulse Sales package, the report from Practical 1 (optional starting point).

Step-by-step procedure

1. Create a new **List** report against the RetailPulse Sales package.
2. Add **Region, Store Name, Product Category, Quantity, Sales Amount** as columns.
3. Select Region, then Ctrl-click Store Name, and choose **Group/Ungroup** to build a two-level hierarchy (Region > Store Name).
4. Select Sales Amount: set **Data Format > Currency**. Select Quantity and Sales Amount and set **Horizontal alignment: Right**.
5. Select the Sales Amount column and click **Summarize > Total** so a footer total is created for each group (and overall).
6. From the Toolbox, drag a **Block/Text item** and **Date** into the **Page header** (or Page > Page Header), enter the report title and today's run date, then **Run** and **Save**.

Expected output (illustrative)

RetailPulse – Sales by Region and Store Run date: 06 October 2026

Region	Store Name	Product Category	Quantity	Sales Amount
East	Boston Central	Electronics	310	$186,000
		Apparel	540	$43,200
		Home & Garden	220	$30,800
		Grocery	1,250	$31,250
	Total Boston Central		**2,320**	**$291,250**
	NYC Midtown	Electronics	405	$243,000
		Apparel	610	$51,850
		Home & Garden	185	$27,750
		Grocery	1,480	$37,000
	Total NYC Midtown		**2,680**	**$359,600**
North	Chicago Loop	Electronics	350	$210,000
		Apparel	500	$40,000
		Home & Garden	260	$36,400
		Grocery	1,320	$33,000
	Total Chicago Loop		**2,430**	**$319,400**
	Minneapolis Mall	Electronics	280	$168,000
		Apparel	455	$36,400
		Home & Garden	240	$33,600
		Grocery	1,100	$27,500
	Total Minneapolis Mall		**2,075**	**$265,500**
Overall Total (all regions)			**19,345**	**$2,494,250**

Mock-up: Region bands containing nested Store bands; duplicates collapsed; currency format; right-aligned numbers; per-store totals. West and South regions follow on later pages.

Expected outcome
Rows grouped by Region and Store Name with duplicates collapsed; Sales Amount in currency format and right-aligned; each store total equals the sum of its rows; the page header (title + date) appears on every page.

Practical 3: Filter RetailPulse Sales by Region and Date

Related topic: Focus Reports Using Filters

Objective

Apply a simple filter and an advanced detail filter to scope a report to a single region and reporting period, then generalize it with a prompt.

Materials required

IBM Cognos Analytics access, the grouped RetailPulse list report from Practical 2.

Step-by-step procedure

1. Open the RetailPulse list report built in Practical 2.
2. Open the **Filters** pane (Filter icon) and add a simple filter: **Region = "North"**.
3. Add an advanced detail filter (**Filters > Add > Advanced**, Detail filter, *Before aggregation*) with the expression: `[Sales Date] between 2026-10-01 and 2026-12-31` (the current quarter start and end dates).
4. Combine both conditions with **AND** (set the connector between the two filters to AND in the Filters pane / Advanced expression).
5. **Run** the report and confirm the row count drops to match the filtered scope (e.g. 32 rows → 8 rows).
6. Select the Region filter, change **Prompt: Ask every time** (i.e. convert to a prompted filter), so other managers can reuse it. Re-run and choose "South" to test.

Combined filter expression

```
[Region] = 'North' AND [Sales Date] BETWEEN 2026-10-01 AND 2026-12-31
```

Prompted version (Region): `[Region] = ?Region? AND [Sales Date] BETWEEN 2026-10-01 AND 2026-12-31`

Expected output (illustrative, Q4-to-date figures)

RetailPulse – Sales by Region and Store Filter: Region = North, Oct–Dec 2026

Region	Store Name	Product Category	Quantity	Sales Amount
North	Chicago Loop	Electronics	29	$17,500
North	Chicago Loop	Apparel	41	$3,333
North	Chicago Loop	Home & Garden	21	$3,033
North	Chicago Loop	Grocery	110	$2,750
North	Minneapolis Mall	Electronics	23	$14,000
North	Minneapolis Mall	Apparel	37	$3,033
North	Minneapolis Mall	Home & Garden	20	$2,800
North	Minneapolis Mall	Grocery	91	$2,291

Mock-up: only North rows for the current quarter remain (8 rows vs 32 unfiltered). Re-running the prompted version with 'South' returns Atlanta Peachtree and Miami Beach rows instead.

Expected outcome
Only rows satisfying both conditions remain; the row count is visibly smaller than the unfiltered report; the prompted version returns a different region's data when re-run with a new value.

Practical 4: Build a RetailPulse Region × Product Crosstab

Related topic: Create Crosstab Reports

Objective

Construct a crosstab report summarizing RetailPulse sales across Region and Product Category with multiple measures and summary totals.

Materials required

IBM Cognos Analytics access, RetailPulse Sales package.

Step-by-step procedure

1. Start a new report and choose the **Crosstab** template.
2. Drag **Region** into the **Rows** edge and **Product Category** into the **Columns** edge.
3. Drag **Sales Amount** into the **Measure** (body) area of the crosstab.
4. Drag **Quantity** next to Sales Amount in the column edge (nested under each Product Category) as a second measure.
5. Select Region and Product Category and choose **Summarize > Total** (or Calculate > Total) to add row and column grand totals.
6. Select Sales Amount, set **Data Format > Currency**, **Run**, and save as **"RetailPulse Region-Product Crosstab"**.

Expected output (illustrative)

RetailPulse Region-Product Crosstab

	Electronics		Apparel		Home & Garden		Grocery		Total	
Region	**Sales**	**Qty**	**Sales**	**Qty**	**Sales**	**Qty**	**Sales**	**Qty**	**Sales**	**Qty**
East	$429,000	715	$95,050	1,150	$58,550	405	$68,250	2,730	**$650,850**	**5,000**
North	$378,000	630	$76,400	955	$70,000	500	$60,500	2,420	**$584,900**	**4,505**
South	$375,000	625	$99,100	1,200	$71,400	510	$65,000	2,600	**$610,500**	**4,935**
West	$417,000	695	$91,800	1,080	$74,200	530	$65,000	2,600	**$648,000**	**4,905**
Total	**$1,599,000**	**2,665**	**$362,350**	**4,385**	**$274,150**	**1,945**	**$258,750**	**10,350**	**$2,494,250**	**19,345**

Mock-up: grand total Sales $2,494,250 and Quantity 19,345 match the Overall Total in the Practical 2 list report (reconciliation check).

Expected outcome
Regions down, product categories across, each cell showing summed Sales Amount and Quantity; row and column totals reconcile with the list report; Sales Amount is consistently formatted as currency.

Practical 5: Build the RetailPulse Sales Prompt Report

Related topic: Design a Cognos Prompt Report for Sales Analysis

Objective

Design a prompt page with value, date-range, and cascading prompts, and link each control to a report filter.

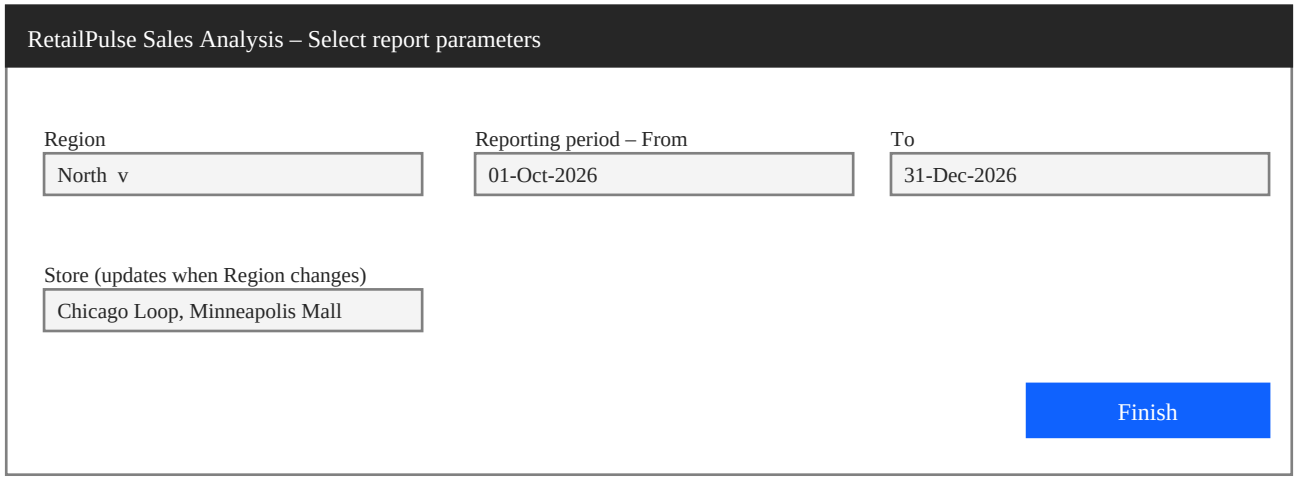
Materials required

IBM Cognos Analytics access, the RetailPulse Region-Product Crosstab from Practical 4.

Step-by-step procedure

1. Open the **RetailPulse Region-Product Crosstab** report.
2. Open the **Pages** tab (Page explorer) and add a **Prompt page** ahead of the crosstab page.
3. Drag a **Value prompt** onto the prompt page; set Parameter = **Region**, Use = [Region], Display = [Region]. Drag a **Date range** (Date & Time) prompt; Parameter = **Period** (range).
4. Add another **Value prompt** for **Store Name** and set its **Cascade source** to the Region prompt, so its list only shows stores of the selected region.
5. On the crosstab page, create filters: `[Region] = ?Region?`, `[Store Name] in (?Store?)`, `[Sales Date] in_range ?Period?` so each prompt is linked to its filter.
6. Set **Default selections** (Region = North; Period = current quarter) on each prompt, add a **Finish** button, then **Run** and test: choose North, then re-run with South.

Expected output (illustrative prompt page)



Mock-up of the prompt page: Region value prompt, date-range prompt, cascading Store prompt, Finish button. Changing Region to South changes the store list to Atlanta Peachtree / Miami Beach.

Expected outcome
All three controls render; changing Region refreshes the Store list without error; Finish returns a crosstab filtered to the chosen values; defaults produce a sensible report with no changes.

Practical 6: Build the RetailPulse Customer Insights Dashboard

Related topic: Build a Dashboard for Customer Insights Visualization

Objective

Assemble a dashboard combining a summary card, trend chart, segment breakdown, and a shared filter widget for RetailPulse customer data.

Materials required

IBM Cognos Analytics access, RetailPulse Customers data module.

Step-by-step procedure

1. Select **New > Dashboard**, choose a template/layout, then **Select a source** and pick the **RetailPulse Customers** data module.
2. Add a **Summary** visualization (KPI card) with a count of **Active Customers** (filtered to Active = Yes and the selected period).
3. Add a **Line chart**: X axis = Month, Y axis = Customer Count, colour/series = Customer Type (New vs Returning).
4. Add a **Bar chart**: X axis = Loyalty Segment, Y axis = Customer Count.
5. Add a **Selection control** (filter widget) on Region; in the Filters/Connections settings make sure it applies to all widgets (cross-filtering on).
6. Arrange widgets — filter at top, KPI card top-left, trend line wide in the middle, bar chart beside/below it — remove overlaps, then **Save** as "RetailPulse Customer Insights".

Expected output (illustrative dashboard)



Mock-up with sample numbers: KPI card, new-vs-returning trend line, loyalty-segment bar chart and a Region filter widget that cross-filters all three. (Segment bars sum to the KPI value, 48,215.)

Expected outcome
All widgets show live customer data (no placeholders); selecting a region updates all widgets consistently; the trend line spans the full period; the layout has a clear hierarchy with no overlaps.

Practical 7: Import and Transform RetailPulse Data in Power BI

Name:		Roll No.:		Date:	

Objective

Import the RetailPulse workbook into Power BI Desktop and clean/shape it with Power Query.

Steps

1. Identify Report, Table and Model views, the ribbon, and the Data, Visualizations and Filters panes.

2. Home > Get data > Excel workbook > tick **Sales, Product, Store** > **Transform Data**.

3. Set types: Sales Date = Date, Quantity = Whole Number, Sales Amount = Fixed decimal number.

4. Remove blank rows and duplicates, replace null Quantity with 0, rename unclear headers (Qty → Quantity, Sales Amt → Sales Amount).

5. Add Conditional Column **Sale Size**: Sales Amount >= 1000 → Large, else Standard.

6. Check Applied Steps, **Close & Apply**, save as **RetailPulse_Analysis.pbix**.

Power Query (M) code

```
= Table.AddColumn(#"Replaced Value", "Sale Size",
    each if [Sales Amount] >= 1000 then "Large" else "Standard", type text)
```

Applied Steps: Source → Navigation → Changed Type → Removed Blank Rows → Removed Duplicates → Replaced Value → Added Conditional Column.

Output

Check	Before	After
Rows	157	150
Blank rows	3	0
Duplicate rows	4	0
Null Quantity	5	0
Sale Size	-	Large 46, Standard 104

Output generated from a sample RetailPulse dataset; replace with your own Power BI screenshots/values.

Paste your Power BI screenshot here: Applied Steps pane

Result

All three tables load without errors, types are correct, blank/duplicate rows are gone, Sale Size labels every row and Applied Steps re-runs on refresh.

Viva-Voce Answers

a) Import vs DirectQuery?

Import copies data into Power BI's in-memory model (fast, full features, refresh needed); DirectQuery queries the source live (always current, slower, limited features).

b) Applied Steps?

Ordered, editable record of every transformation (M code), replayed automatically on refresh.

c) View for relationships?

Model view.

d) Why set data types first?

Types control allowed operations, sorting, date features and relationships; wrong types cause errors or wrong totals.

Practical 8: Model, Calculate, and Publish a RetailPulse Sales Report

Name:		Roll No.:		Date:	

Objective

Create relationships, write basic DAX, build interactive visuals and publish to the Power BI service.

Steps

1. Model view: one-to-many (single direction) Product[ProductID] → Sales[ProductID] and Store[StoreID] → Sales[StoreID].

2. Create measures Total Sales, Total Quantity, Average Order Value.

3. Add calculated column Sales per Unit.

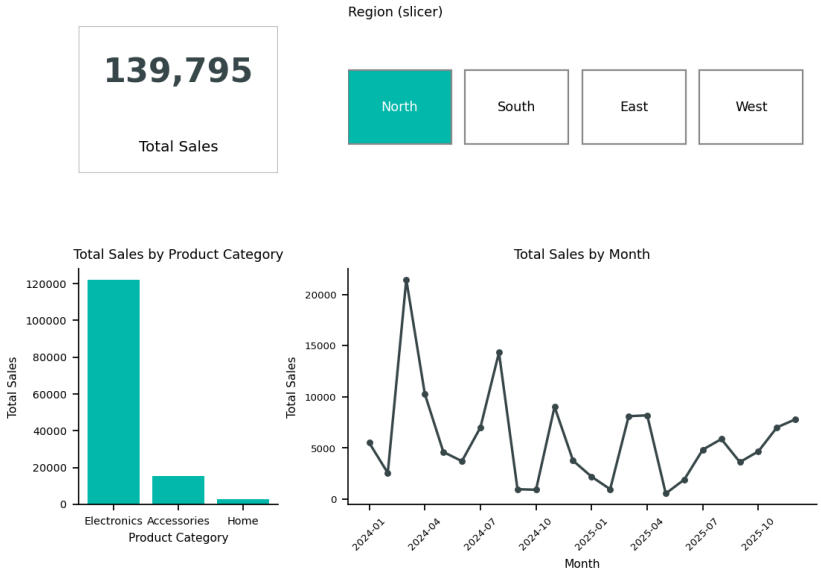
4. Report page: card (Total Sales), bar chart by Product Category, line chart by month, Region slicer.

5. Test cross-filtering; add titles and axis labels.

6. Home > Publish > choose workspace; open in the service and share with instructor.

```
Total Sales = SUM(Sales[Sales Amount])
Total Quantity = SUM(Sales[Quantity])
Average Order Value = DIVIDE([Total Sales], DISTINCTCOUNT(Sales[OrderID]))
Sales per Unit = DIVIDE(Sales[Sales Amount], Sales[Quantity])   -- calculated column
```

Output



Total Sales	Total Quantity	Average Order Value	Region slicer: N / S / E / W
139,794.72	435	931.96	27,754 / 30,295 / 47,709 / 34,037

Output generated from a sample RetailPulse dataset; replace with your own Power BI screenshots/values.

Paste your Power BI screenshot here: Model view and published report

Result

Product and Store connect to Sales with 1 and * markers; measures respond to the Region slicer; the card equals the sum of the bars; the report opens in the service for users with access.

Viva-Voce Answers

a) Why one-to-many from dimension to fact?

Unique dimension keys filter fact rows unambiguously (star schema), avoiding duplicates and saving memory.

b) Measure vs calculated column?

Column: computed per row at refresh and stored. Measure: computed at query time in the current filter context, not stored.

c) Why DIVIDE?

It returns blank/alternate result on divide-by-zero instead of an error.

d) Trend visual?

Line chart: shows direction and change over continuous time.

e) Needed to view a published report?

Published to a workspace, viewer has access (role/share) and a suitable licence; RLS role if applicable.

Practical 9: Time Intelligence and Advanced DAX for RetailPulse

Name:		Roll No.:		Date:	

Objective

Create a date table, build time-intelligence and context-modifying DAX measures, and use Performance Analyzer.

Steps

1. Create Date table, add Year, Month Number, Month Name; mark as date table; relate Date[Date] to Sales[Sales Date].

2. Create Sales YTD and Sales LY.

3. Create YoY Growth % with VAR/RETURN.

4. Create Region Share % using CALCULATE + ALL.

5. Build matrix: Year/Month rows; Total Sales, Sales YTD, Sales LY, YoY Growth % values.

6. Performance Analyzer: record, note durations, remove unused columns and visuals, record again.

```
Date = CALENDAR(MIN(Sales[Sales Date]), MAX(Sales[Sales Date]))
Sales YTD = TOTALYTD([Total Sales], 'Date'[Date])
Sales LY  = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
YoY Growth % = VAR Curr = [Total Sales] VAR Prev = [Sales LY] RETURN DIVIDE(Curr - Prev, Prev)
Region Share % = DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Store[Region])))
```

Output (excerpt: year boundary)

Year	Month	Total Sales	Sales YTD	Sales LY	YoY Growth %
2024	Nov	9,025	80,330		
2024	Dec	3,755	84,085		
2025	Jan	2,189	2,189	5,501	-60.2%
2025	Feb	970	3,159	2,560	-62.1%
2025	Mar	8,103	11,262	21,446	-62.2%
2025	Apr	8,185	19,447	10,273	-20.3%

Region Share %	East	North	South	West	Total
	34.1%	19.9%	21.7%	24.3%	100.0%

Sample figures; Performance Analyzer timings must be recorded by you (compare first and second recording).

Paste your Power BI screenshot here: Matrix and Performance Analyzer

Result

YTD resets each January; first-year Sales LY and YoY are blank; Region Share % totals 100%; Performance Analyzer shows per-visual timings and the second recording is equal or faster.

Viva-Voce Answers

a) Why mark a date table?

Time intelligence needs a continuous, unique date column; marking it identifies it and avoids auto date/time tables.

b) What does CALCULATE do?

Evaluates an expression after modifying the filter context.

c) Why VAR/RETURN?

Evaluates once, improves readability and performance.

d) Role of ALL?

Removes the Region filter from the denominator so share = region / grand total.

Practical 10: Interactive Dashboard, Row-Level Security, and Power Platform Integration

Name:		Roll No.:		Date:	

Objective

Build an interactive report, restrict data by region with RLS, and connect to Power Automate and Power Apps.

Steps

1. Add Year and Region slicers, a drill-through page **Store Detail** (field: Store Name) and two bookmarks with buttons (chart view / table view).

2. Modeling > Manage roles: create **North Manager** and **South Manager** with DAX filters on Store.

3. Test each role with **View as**.

4. Publish; in the service assign test accounts to roles (dataset > Security); pin the Total Sales card to a dashboard.

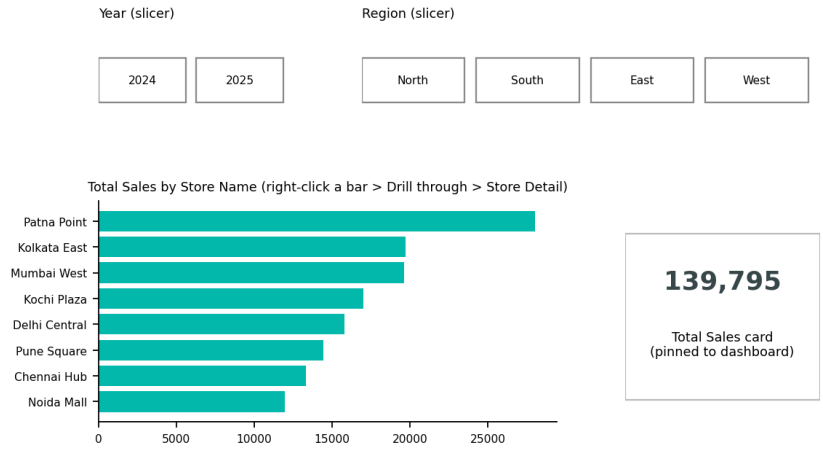
5. Set a data alert on the card; create a Power Automate flow (trigger: *When a data driven alert is triggered*) that sends an email/Teams message.

6. Add a Power Apps visual on Store Detail (Store Name, Sales Amount) > Create new > app shows selected store.

```
North Manager:  Store[Region] = "North"
South Manager:  Store[Region] = "South"
```

Output

View as	Regions visible	Total Sales	Orders
No role	East, North, South, West	139,794.72	150
North Manager	North	27,753.84	34
South Manager	South	30,294.89	36



Output generated from a sample RetailPulse dataset; replace with your own Power BI screenshots/values.

Paste your Power BI screenshot here: View as result / flow run

Result

Slicers, drill-through and bookmarks work; each manager sees only their region; the alert triggers the flow and a notification arrives; the Power Apps visual updates with the selected store. (Steps 4-6 need Pro/PPU and a work/school account.)

Viva-Voce Answers

a) What is RLS and how are roles defined?

RLS limits which rows a user sees. Roles are created under Modeling > Manage roles with DAX table filters (e.g. [Region] = "North"), then users are assigned in the service's dataset Security.

b) How to test RLS?

Use View as in Desktop and Test as role in the service.

c) What is drill-through?

A detail page that receives the selected item (store) as a filter.

d) Role of Power Automate / Power Apps?

Automate runs workflows from alerts (email/Teams); Power Apps embeds an app using the selected report context.